# RDFProxy: A Model-Centric Approach to Transforming SPARQL Result Sets for Linked Data Clients

**Plank, Lukas**
lukas.plank@oeaw.ac.at
Austrian Academy of Sciences, Austria

**Wünsche, Katharina**
katharina.wuensche@oeaw.ac.at
Austrian Academy of Sciences, Austria

## 1 Introduction

Knowledge Graphs and Linked Data have become invaluable tools in the Digital Humanities (DH), enabling the representation of complex, interconnected datasets essential to the various fields of DH research. DH Knowledge Graphs typically utilize sophisticated ontologies, such as CIDOC CRM and CRM–compatible models, which are tailored to capture the often intricate domain knowledge of humanities scholarship. However, despite their value, integrating DH Knowledge Graphs into client applications, such as web frontends, and backend APIs serving those clients, presents significant challenges.[1]

SPARQL, the W3C-standard query language and specification for retrieving data from Knowledge Graphs is designed primarily for pattern-based retrievals, delivering results of whatever binding projections satisfy a given query pattern in flat, basically row-based structures. In contrast, modern RESTful API consumers typically expect potentially deeply nested JSON shapes, hierarchical object groupings, and seamless integration with pagination and documentation standards. These expectations underscore a considerable gap between the structure of SPARQL result sets and the requirements of contemporary API design.

This paper introduces RDFProxy (RDFProxy 2024), a Python library designed to bridge the gap between the inherently flat solution sets of SPARQL responses and the rich, nested JSON models required by modern API consumers. To this end, RDFProxy implements a mapping mechanism that allows the projection of SPARQL result rows onto Pydantic (Pydantic 2024) models, which provide a robust modeling abstraction built on top of Python's type annotation system.

By choosing Pydantic as the modeling grammar for RDFProxy, developers not only gain the ability to express and validate typed and nested data models but also benefit from seamless integration with FastAPI, a modern, high performance web framework for building APIs with Python. This integration allows RDFProxy-based backends to leverage the powerful capabilities of Pydantic and FastAPI for RDF data sets, including model pre- and post-validation hooks, along with custom data serializers, native support for asynchronous API calls and automatic generation of detailed API documentation according to the OpenAPI standard (OpenAPI Specification 2024).

## 2 Related Work

Prior approaches in developing REST APIs on top of SPARQL endpoints have resulted in various tools and frameworks. Ramose (Daquino et al. 2022) provides a lightweight wrapper for SPARQL endpoints, allowing simplified RESTful interactions but with limited capabilities for nested outputs or paginated result lists. Similarly, BASIL (Daga et al. 2015) allows users to define SPARQL queries and query parameters that are provided via a REST API and a corresponding OpenAPI description. Schröder et al. (Schröder et al. 2018) follow a "zero-configuration" approach by directly transforming RDF graphs into path-based APIs, similar to SPARQL property paths. Grlc (Meroño-Peñuela / Hoekstra 2016) automates the creation of RESTful APIs by leveraging SPARQL queries stored in GitHub repositories, which simplifies version control and API adaptations but does not support applying transformations to the query results. SPARQL Transformer (Lisena / Troncy 2018) enables the transformation of SPARQL results into customizable JSON or JSON-LD structures using a custom query syntax at the expense of requiring query adaptations.

A synopsis of the features provided by the aforementioned work can be found in table 1. The comparison illustrates that each approach has its own focus, with RDFProxy currently being the only solution meeting all of the above criteria. It should also be mentioned that, while SPARQL Transformer provides grouping and pagination functionality, these transformations are applied in a post-processing step after SPARQL query execution, which potentially imposes significant performance penalties, particularly for larger graphs.

*Table 1: Overview of related work compared to RDFProxy*

# 3 RDFProxy: Overview and Practical Use

The following provides a terse general overview of the functionality and design of RDFProxy as well as an example of implementing a simple RDFProxy-based FastAPI route to retrieve, reshape and relay (i.e. proxy) data from a remote SPARQL endpoint, illustrating how the RDFProxy mapping mechanism enables the projection of SPARQL result sets onto nested and grouped data shape definitions.

The primary code interface for defining API routes with RDFProxy is the *SPARQLModelAdapter* class, which is designed to establish and handle a connection between a triplestore, a SPARQL query and a Pydantic model on a perroute basis. By associating a *SPARQLModelAdapter* instance with each route, RDFProxy suggests a modular and flexible approach to defining endpoints, allowing the modification and extension of individual routes without affecting the rest of the API and also enabling a single RDFProxy-based backend to potentially connect with multiple SPARQL endpoints.

Based on these parameters (endpoint, query, model), the *SPARQLModelAdapter* employs a query constructor mechanism to dynamically modify the incoming SPARQL query - for instance, by injecting certain clauses and subqueries to implement purely SPARQL-based pagination. The modified query is then executed against the triplestore's endpoint, and the resulting SPARQL response is delegated to a mapping component, which is responsible for reshaping the SPARQL result row data by projecting it onto the given Pydantic model definition.


```
{
  "items": [
    {
      "gnd": "119359464",
      "surname": "Schindel",
      "works": [
        {
          "name": "Geb\u00fcrtig",
          "viafs": []
        }
      ]
    },
    {
      "gnd": "115612815",
      "surname": "Geiger",
      "works": [
        {
          "name": "Der alte K\u00f6nig in seinem Exil",
          "viafs": [
            "299260555",
            "676215438"
          ]
        },
        {
          "name": "Unter der Drachenwand",
          "viafs": [
            "22771517170533139"
          ]
        }
      ]
    }
  ],
  "page": 1,
  "size": 100,
  "total": 2,
  "pages": 1
}
```


*Figure 1: SPARQLModelAdapter Architecture: Dynamic Query Construction and Result Mapping in RDFProxy*

The RDFProxy shapes mapping facility handles the instantiation of Pydantic model objects by mapping the SPARQL result row bindings to corresponding Pydantic model fields. By default, SPARQL variable names are mapped directly to Pydantic field names, with aliasing being supported through explicit annotation of a field type using *rdfproxy.SPARQLBinding*. For fields typed as lists of Pydantic *BaseModel* subclasses, RDFProxy triggers an aggregation mechanism, grouping applicable models into arrays. In that case, the *group_by* configuration parameter, defined within *model_config*, determines the model field by which the list items should be grouped. For list fields parameterized with non-model (scalar) types, such as *list[str]*, the mapping process gathers multiple values into arrays, again based on the *group_by* configuration parameter.

Building on this brief technical overview, the remainder of the section showcases a practical implementation of an RDFProxy-based FastAPI route, highlighting the role of shape definitions using Pydantic models to structure and organize the retrieved SPARQL result sets.

The example query below uses a VALUES clause to statically define a solution sequence and thus retrieves a set of four result rows expressing basic author/work data from any SPARQL 1.1 endpoint:

```
{
  "items": [
    {
      "gnd": "119359464",
      "surname": "Schindel",
      "works": [
        {
          "name": "Geb\u00fcrtig",
          "viafs": []
        }
      ]
    },
    {
      "gnd": "115612815",
      "surname": "Geiger",
      "works": [
        {
          "name": "Der alte K\u00f6nig in seinem Exil",
          "viafs": [
            "299260555",
            "676215438"
          ]
        },
        {
          "name": "Unter der Drachenwand",
          "viafs": [
            "22771517170533139"
          ]
        }
      ]
    }
  ],
  "page": 1,
  "size": 100,
  "total": 2,
  "pages": 1
}
```


*Listing 1: Static SPARQL query retrieving basic author/work data*


```
{
  "items": [
    {
      "gnd": "119359464",
      "surname": "Schindel",
      "works": [
        {
          "name": "Geb\u00fcrtig",
          "viafs": []
        }
      ]
    },
    {
      "gnd": "115612815",
      "surname": "Geiger",
      "works": [
        {
          "name": "Der alte K\u00f6nig in seinem Exil",
          "viafs": [
            "299260555",
            "676215438"
          ]
        },
        {
          "name": "Unter der Drachenwand",
          "viafs": [
            "22771517170533139"
          ]
        }
      ]
    }
  ],
  "page": 1,
  "size": 100,
  "total": 2,
  "pages": 1
}
```


*Listing 2: RDFProxy/Pydantic model definition for Author/Work data*

Note how this illustrates the previously outlined problem of SPARQL result sets representing conceptually composed (or generally *composable*) information as flat binding mappings: Two distinct author entities associated with three distinct work entities, where the same work may have multiple VIAF identifiers asserted about it, are retrieved as four partially redundant result rows.

Given the result rows retrieved by the example query, to represent the shape of an Author entity associated with one or more Work entities, where each Work can have multiple values for a single field, one can come up with the following Pydantic model definition:

Here, the *Author* model defines three fields: the *gnd* field corresponds directly to a name in the set of SPARQL bindings and will be mapped accordingly; the *surname* field is explicitly allocated using the *rdfproxy.SPARQLBinding* aliasing feature, i.e. the value of the SPARQL variable *nameLabel* will be assigned to the *surname* field in the model; and the *works* field references another model, *Work*, which shall be aggregated into an array based on their common gnd values (as indicated in the model_config). The Work model is defined similarly, with the *viafs* field aggregating a scalar type and using an *rdfproxy.SPARQLBinding* alias. Based on the above SPARQL query and corresponding Pydantic model, an *rdfproxy.SPARQLModelAdapter* object targeting an arbitrary SPARQL 1.1 endpoint can be instantiated and directly used in a FastAPI route.

```
{
  "items": [
    {
      "gnd": "119359464",
      "surname": "Schindel",
      "works": [
        {
          "name": "Geb\u00fcrtig",
          "viafs": []
        }
      ]
    },
    {
      "gnd": "115612815",
      "surname": "Geiger",
      "works": [
        {
          "name": "Der alte K\u00f6nig in seinem Exil",
          "viafs": [
            "299260555",
            "676215438"
          ]
        },
        {
          "name": "Unter der Drachenwand",
          "viafs": [
            "22771517170533139"
          ]
        }
      ]
    }
  ],
  "page": 1,
  "size": 100,
  "total": 2,
  "pages": 1
}
```


*Listing 3: FastAPI route definition using a SPARQLModelAdapter instance*

FastAPI query parameters (i.e. route function parameters not corresponding to any path parameter definitions) are expected to be passed along to the *SPARQLModelAdapter.query* method,[2] which initiates the RDFProxy query and mapping mechanism and returns an *rdfproxy.Page* instance.

The *rdfproxy.Page* class is itself a composite Pydantic model that encapsulates the generated data shapes (within its *items* field) as well as dynamically computed pagination data, resulting in the JSON-serialized response shown in listing 4. Note that the *rdfproxy.Page* model is also a Python typing generic which enables type parameterization of the route response model on the shapes model; i.e., for instance, the route defined in Listing 3 is type-annotated to return a parameterized generic, *Page[Author]*. Providing a generic response model type not only enhances type safety and static analysis, but also enables Pydantic runtime validation and refined OpenAPI schema and documentation generation, leveraging the powerful type annotation-based features of both Pydantic and FastAPI.

# 4 Conclusion

While significant related work has been conducted in the area of building REST APIs on top of RDF Graphs, currently no existing solution fully meets the specific needs and expectations of contemporary API consumers.

To address this problem, RDFProxy proposes a model-centric approach to bridging the gap between Knowledge Graphs and modern data-driven client applications. By building on robust existing tools like Pydantic and FastAPI, RDFProxy aims to provide a flexible and extensible framework that simplifies the integration of RDF graphs into RESTful services.

Currently, RDFProxy is being developed at the Austrian Centre for Digital Humanities and Cultural Heritage (ACDH-CH) and will serve as the backend solution for the Releven project at University of Vienna. (Andrews et al. 2024) The Releven project, with its intricate and highly reified CRM-based STAR model, presents an excellent and challenging initial trial for RDFProxy's practical capabilities.

By aligning the strengths of RDF Knowledge Graphs with the accessibility of contemporary standards-based REST APIs, RDFProxy aspires to make semantic data sets more accessible, usable, and impactful in the context of Digital Humanities and beyond.


```
{
  "items": [
    {
      "gnd": "119359464",
      "surname": "Schindel",
      "works": [
        {
          "name": "Geb\u00fcrtig",
          "viafs": []
        }
      ]
    },
    {
      "gnd": "115612815",
      "surname": "Geiger",
      "works": [
        {
          "name": "Der alte K\u00f6nig in seinem Exil",
          "viafs": [
            "299260555",
            "676215438"
          ]
        },
        {
          "name": "Unter der Drachenwand",
          "viafs": [
            "22771517170533139"
          ]
        }
      ]
    }
  ],
  "page": 1,
  "size": 100,
  "total": 2,
  "pages": 1
}
```


*Listing 4: JSON response from the FastAPI route defined in Listing 3*

# Notes

1. This difficulty of KG/UI integration is especially problematic in Digital Humanities contexts, where user interfaces ideally serve as academically curated mediations between users and data; e.g. Scholarly Editions can in that sense be understood as "critically argued interface[s] to the text" (Andrews / Zundert 2014). This perspective on interfaces as "argumentative utilities" also shifts the representation/presentation dichotomy towards more of an interdependency relationship, enabling an understanding of interfaces as semiotically significant, rule-based, documented and particularly informed interpretive acts that constitute an integral part of the argument a Knowledge Graph makes about its represented data. (Andrews / Zundert 2018)
2. Currently, only the required *page* and *size* query parameters are supported. Planned features are e.g. ordering and complex filtering, which should be implemented using FastAPI Query Model Parameters. See https://fastapi.tiangolo.com/tutorial/query-param-models/

# Bibliography


Andrews, Tara / Rózsa, Márton / Prajda, Katalin / Read, Lewis / Anđelović, Aleksandar (2024): “Re-Evaluating the Eleventh Century through Linked Events and Entities”. In: Historical Studies on Central Europe 4.1, pp. 217–245. doi: 10.47074/HSCE.2024-1.12. url: https://doi.org/10.47074/HSCE.2024-1.12 .

Andrews, Tara / Zundert, Joris J. van (2014): “Apparatus vs Graph - an Interface as Scholarly Argument”. In: Graph - an Interface as Scholarly Argument. Interface Critique. Berlin.

Andrews, Tara / Zundert, Joris J. van (2018): “What Are You Trying to Say? The Interface as an Integral Element of Argument”. In: Digital Scholarly Editions as Interfaces. Ed. by Roman Bleier et al. Books on Demand. Chap. 1, pp. 3–33. isbn: 9783748109259.

Daga, Enrico / Panziera, Luca / Pedrinaci, Carlos (2015): “A BASILar approach for building web APIs on top of SPARQL endpoints”. In: CEUR Workshop Proceedings. Vol. 1359, pp. 22–32.

Daquino, Marilena / Heibi, Ivan / Peroni, Silvio / Shotton, David (2022): “Creating RESTful APIs over SPARQL endpoints using RAMOSE”. In: Semantic Web 13.2, pp. 195–213.

Lisena, Pasquale / Troncy, Raphaël (2018): “Transforming the JSON output of SPARQL queries for linked data clients”. In: Companion Proceedings of the The Web Conference 2018, pp. 775–780.

Meroño-Peñuela, Albert / Hoekstra, Rinke (2016): “grlc Makes GitHub Taste Like Linked Data APIs”. In: The Semantic Web. Ed. by Harald Sack et al. Cham: Springer International Publishing, pp. 342–353. isbn: 978-3-319-47602-5.

OpenAPI Specification (2024). Accessed: 2024-11-22. url: https://swagger.io/specification/ .

Pydantic (2024). Accessed: 2024-10-27. url: https://pydantic-docs.helpmanual.io/ .

RDFProxy (2024). Accessed: 2024-10-27. url: https://github.com/acdh-oeaw/rdfproxy .

Schröder, Markus / Hees, Jörn / Bernardi, Ansgar / Ewert, Daniel / Klotz, Peter / Stadtmüller, Steffen (2018): “Simplified SPARQL REST API: CRUD on JSON Object Graphs via URI Paths”. In: The Semantic Web: ESWC 2018 Satellite Events: ESWC 2018 Satellite Events, Heraklion, Crete, Greece, June 3-7, 2018, Revised Selected Papers 15. Springer, pp. 40–45.